

Estruturas em Árvores Avançadas

*Trabalho desenvolvido para a disciplina de Algoritmos e Estruturas de Dados II.

Anthony Gabriel Sotto Mayor Silva

Graduando em Eng. de Computação

CEFET-MG, Campus V

Divinópolis, Brasil

anthonygabriel sotto29@gmail.com

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

Contextualização e Motivação

A organização e a recuperação eficiente de dados são pilares do desenvolvimento de sistemas computacionais. Tradicionalmente, estruturas elementares como a Árvore Binária de Busca (BST) e a Árvore AVL são utilizadas como modelos fundamentais para armazenar, ordenar e buscar itens em memória. Contudo, esses modelos clássicos possuem limitações em cenários operacionais específicos: a BST é estruturalmente vulnerável à degeneração (tornando-se uma lista encadeada) quando submetida a dados previamente ordenados, o que eleva sua complexidade para $O(n)$. Por outro lado, a AVL previne essa degeneração, mas impõe um alto custo computacional contínuo para manter seu balanceamento estrito por meio de rotações a cada operação. Para superar esses gargalos, torna-se necessário o estudo de estruturas em árvores alternativas e especializadas, desenhadas para processamento de cadeias de caracteres, heurísticas de balanceamento dinâmico e particionamento espacial multidimensional.

Árvores Baseadas em Prefixos (Trie e Patricia)

Diferenciando-se das árvores BST e AVL, que determinam a direção espacial comparando chaves inteiras, estas estruturas processam os dados iterativamente caractere por caractere, sendo a solução ideal para dicionários e sistemas de autocompletar.

- **Árvore Trie:** É uma estrutura hierárquica que explora os prefixos compartilhados entre as chaves textuais. A anatomia de seu nó consiste em um vetor fixo de ponteiros (dimensionado para o alfabeto em uso, como 26 posições) e um marcador booleano de fim de palavra. Essa arquitetura garante transições entre nós em tempo constante $O(1)$, mas gera um alto consumo de memória devido à alocação de estruturas e ponteiros nulos.
- **Árvore Patricia:** Projetada para solucionar a ineficiência de memória espacial da Trie por meio da representação compacta de strings. A anatomia de seu nó armazena *substrings* inteiras em vez de caracteres isolados. Sua invariante dita que nós sucessivos sem bifurcação (caminhos de filho único) devem ser mesclados, ocorrendo um processo de divisão de prefixos (*split*) apenas no momento exato em que novas ramificações divergem. É amplamente aplicada em indexação compacta e roteamento de redes.

Árvores de Balanceamento Dinâmico (Splay e Treap)

Estas estruturas abandonam as invariantes rígidas da AVL em favor de adaptações heurísticas e propriedades probabilísticas para manterem a estabilidade da árvore.

- **Árvore Splay:** Uma árvore autoajustável que reorganiza dinamicamente sua estrutura após qual-

quer operação de acesso. Ela utiliza rotações em cascata para aproximar os elementos mais frequentemente consultados da raiz, tirando proveito da localidade temporal de referência em bases de dados com acessos desiguais.

- **Árvore Treap:** Estrutura híbrida onde a anatomia do nó abriga duas informações fundamentais: uma chave e um valor de prioridade. As chaves obedecem à regra de ordenação da BST, enquanto as prioridades (geradas de forma aleatória) impõem a invariante de um Heap. Através de rotações locais focadas nas prioridades, a Treap alcança um balanceamento probabilístico médio de $O(\log n)$, resistindo ao pior caso estrutural com uma implementação essencialmente mais simples que a AVL.

Particionamento Espacial Multidimensional (KD-Tree)

- **KD-Tree:** Enquanto as árvores tradicionais organizam chaves unidimensionais, a KD-Tree é estruturada para organizar e buscar pontos de coordenadas em múltiplos eixos espaciais. A anatomia de seus nós armazena vetores dimensionais. Sua invariante exige a alternância da dimensão de comparação (ex: eixos X e Y) a cada nível de profundidade da árvore durante o particionamento espacial. Essa propriedade matemática permite o descarte em massa de subárvores inteiras (*pruning*) durante a busca pelo vizinho mais próximo, justificando sua escolha para sistemas de geometria computacional e ferramentas GIS.

Discussão e Diferenças Conceituais

O desenvolvimento computacional dessas cinco estruturas evidencia as lacunas deixadas pelas BSTs e AVLs. A Trie e a Patricia eliminam a dependência de comparações completas de chaves, vinculando a complexidade diretamente ao comprimento da entrada, e não à volumetria da base de dados. A Splay e a Treap, por sua vez, comprovam que é possível

alcançar estabilidade computacional sem depender da rigidez analítica dos algoritmos de balanceamento tradicionais (como os da AVL), ainda que por caminhos distintos: enquanto a Splay reorganiza dinamicamente sua estrutura em função da frequência de acesso, aproximando da raiz os elementos mais consultados, a Treap alcança um balanceamento probabilístico por meio de prioridades aleatórias atribuídas na inserção, independentemente do padrão de uso posterior. Em suma, as diferenças conceituais dessas árvores representam a transição entre modelos genéricos de ordenação para ferramentas de indexação desenhadas para arquiteturas e problemas de nicho.

II. PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

A. Árvore Patricia (*Radix Tree Compacta*)

Representação Computacional e Anatomia dos Nós

Diferente das árvores baseadas em comparações numéricas clássicas, a Árvore Patricia organiza a informação hierarquizando prefixos textuais. Sua representação computacional foi desenhada para mitigar o desperdício de memória das estruturas de prefixo convencionais (como a Trie). A anatomia do seu nó agrupa caracteres consecutivos em uma variável `std::string`, além de possuir um vetor de ramificações e um identificador de validade.

```
// Anatomia do nó compactado
struct NoPatricia {
    std::string prefixo;
    bool fimPalavra;
    NoPatricia* filhos[TAM_ALFABETO];
};
```

Organização das Informações e Decisões de Projeto

A principal decisão de projeto da estrutura é a **compactação de caminhos**: nós sucessivos que não apresentam bifurcação são obrigatoriamente mesclados. Essa decisão exige que a entrada de dados seja rigorosamente validada (`indice < 0`

|| indice >= TAM_ALFABETO) para impedir falhas de segmentação ao acessar o vetor estático.

Operações Fundamentais e Manutenção Estrutural

A manutenção da propriedade compacta exige que as operações manipulem ativamente a topologia da árvore.

- **Busca (Correspondência de Blocos):** A operação de consulta consome a palavra iterativamente, comparando-a diretamente com a `std::string` `prefixo` do nó atual. Essa estratégia minimiza os saltos de ponteiros na memória, otimizando o tempo de processamento. O trecho essencial abaixo demonstra a busca pelo Maior Prefixo Comum (LCP) dentro do nó:

```
// Busca consumindo blocos de caracteres
while (j < prefixoNo.length() && i <
↳ palavra.length() &&
    prefixoNo[j] == palavra[i]) {
    operacoes++; // Conta a comparacao direta
    ↳ na string
    i++; j++;
}
if (j < prefixoNo.length()) return false; //
↳ Divergiu no meio
```

- **Percurso e Consulta:** A implementação atual não contempla uma operação de percurso dedicada; a extração de todas as chaves armazenadas seria possível por meio de um DFS sobre o vetor de filhos, concatenando os prefixos acumulados no caminho, mas essa operação não foi implementada neste trabalho.

- **Inserção e Divisão (Split):** O mecanismo central de estabilidade ocorre quando uma nova chave diverge estruturalmente de um prefixo existente. O algoritmo invoca a divisão do nó (*Split*), isolando o prefixo comum no pai e bifurcando os fragmentos restantes.

```
ALGORITMO Inserir_Patricia(no_atual,
↳ nova_palavra):
    prefixo_comum <- comparar(no_atual.prefixo,
    ↳ nova_palavra)

// Manutenção estrutural: Rompimento do nó
↳ (Split)
```

SE a divergencia ocorrer no meio do

```
↳ prefixo ENTAO:
    novo_no_split <- CriarNo(resto do
    ↳ prefixo antigo)
    novo_no_split.fimPalavra <-
    ↳ no_atual.fimPalavra // Herda flag
```

```
// Quebra manual de linha para nao
↳ vaziar a coluna:
Transferir ponteiros de filhos do
↳ no_atual
para o novo_no_split
```

```
no_atual.prefixo <- prefixo_comum
no_atual.fimPalavra <- FALSO
no_atual.adicionarFilho(novo_no_split)
```

```
SE nova_palavra possuir resto nao
↳ consumido:
```

```
↳ no_atual.adicionarFilho(CriarNo(resto
↳ nova_palavra))
```

SENAO:

```
no_atual.fimPalavra <- VERDADEIRO
↳ // Divergencia exata
```

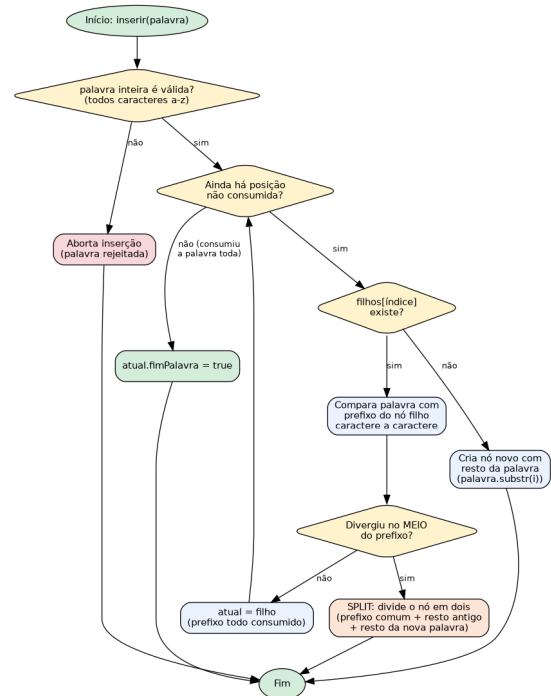


Figura 1. Fluxograma do processo de decisão e divisão (Split) durante a inserção na Árvore Patricia.

- **Remoção e Re-compactação (Merge):** A remoção segue a exclusão lógica, alterando a *flag* `fimPalavra` para falso. Contudo, se essa

invalidação deixar o nó alvo com apenas um filho, a invariante de compactação é violada. Aplica-se então a operação de *Merge*, fundindo fisicamente o nó ao seu filho único remanescente.

```
ALGORITMO Remover_Patricia(chave):
    no_alvo <- Buscar_Navegando(chave)
    SE no_alvo NAO ENCONTRADO RETORNAR

    no_alvo.fimPalavra <- FALSO // Remocao
    ↪ Logica

    // Manutencao estrutural: Re-compactacao
    ↪ (Merge)
    SE no_alvo.qtdFilhos == 1 ENTAO:
        filho_unico <-
            ↪ ObterUnicoFilho(no_alvo)

        // Quebra manual para caber na coluna:
        no_alvo.prefixo <- no_alvo.prefixo +
                               filho_unico.prefixo

        no_alvo.fimPalavra <-
            ↪ filho_unico.fimPalavra

        ↪ no_alvo.TransferirFilhosDe(filho_unico)
        LiberarMemoria(filho_unico)
```

B. Árvore Trie (Árvore de Prefixos)

Representação Computacional e Anatomia dos Nós

A estrutura basal da Trie foi projetada privilegiando o tempo de acesso direto em detrimento do consumo espacial. A anatomia do seu nó elimina a necessidade de buscas sequenciais: ela é composta por um arranjo estático de ponteiros (dimensionado exclusivamente para as 26 letras do alfabeto minúsculo) e uma variável booleana de controle. A análise de memória do projeto revelou que cada nó instanciado consome exatos 216 bytes na arquitetura de 64 bits utilizada.

```
// Anatomia estrutural com acesso rápido
struct NoTrie {
    NoTrie* filhos[TAM_ALFABETO];
    bool fimPalavra;
};
```

Organização das Informações e Decisões de Projeto

A transição topológica entre caracteres é resolvida via aritmética de ponteiros ($c - 'a'$), garantindo

que a identificação do próximo ramal tenha custo de tempo constante $O(1)$. A principal limitação estrutural dessa abordagem é a subutilização de memória, já que inúmeros ponteiros permanecem nulos (vazios) em bases de dados onde as palavras não compartilham muitos prefixos.

Operações Fundamentais e Manutenção Estrutural

As operações da Trie independem do volume total de chaves (N), sendo limitadas estritamente pelo comprimento da string de entrada (k).

- **Busca e Inserção:** O algoritmo navega iterativamente ao longo da palavra. Na inserção, a alocação dinâmica de memória ocorre sob demanda, apenas quando o ponteiro correspondente ao índice do caractere é nulo.

```
ALGORITMO Inserir_Trie(raiz, palavra):
    no_atual <- raiz
    PARA CADA caractere 'c' EM palavra:
        indice <- c - 'a'

        SE indice < 0 OU indice >= 26 ENTAO
            ↪ ABORTAR

        SE no_atual.filhos[indice] EH NULO
            ↪ ENTAO:
                no_atual.filhos[indice] <-
                    ↪ CriarNo()

        no_atual <- no_atual.filhos[indice]

    no_atual.fimPalavra <- VERDADEIRO
```

- **Remoção Lógica (Decisão de Projeto Crítica):** A exclusão física de nós (via delete) foi intencionalmente descartada. Como a Trie baseia-se no compartilhamento massivo de prefixos, destruir um caminho corromperia outras strings derivadas. Adotou-se o princípio da **remoção preguiçosa** (*lazy deletion*): o algoritmo encontra a chave e invalida o marcador, garantindo que a remoção custe o mesmo que a busca.

```
ALGORITMO Remover_Trie(raiz, palavra):
    no_atual <- raiz
    PARA CADA caractere 'c' EM palavra:
        indice <- c - 'a'
```

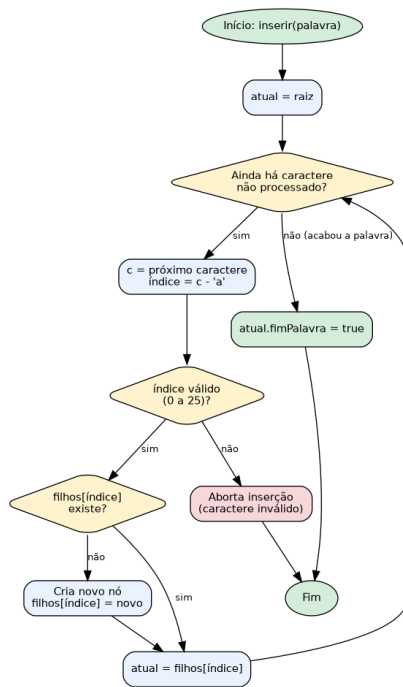


Figura 2. Fluxograma da transição topológica e alocação de nós sob demanda na Árvore Trie.

```

SE indice < 0 OU indice >= 26 ENTAO
  ↳ ABORTAR

// Se o caminho quebra antes do fim, a
  ↳ palavra nao existe
SE no_atual.filhos[indice] EH NULO
  ↳ ENTAO RETORNAR

no_atual <- no_atual.filhos[indice]

// Manutenção Estrutural: Remocao logica
  ↳ sem 'delete'
SE no_atual.fimPalavra EH VERDADEIRO
  ↳ ENTAO:
    no_atual.fimPalavra <- FALSO
  
```

C. Árvore Splay (Árvore de Autoajuste)

Representação Computacional e Anatomia dos Nós

A Árvore Splay é uma árvore de busca binária autoajustável projetada para otimizar sistemas com acessos altamente desiguais ou localidade temporal de referência. A anatomia de seu nó (NoSplay) armazena uma chave numérica inteira e os ponteiros direcionais padrão. O grande diferencial arquitetônico é a presença obrigatória de um ponteiro para

o pai. Essa decisão de projeto permite que as rotações estruturais ocorram de baixo para cima de forma iterativa, sem a necessidade de alocar pilhas auxiliares de recursão na memória.

```

// Trecho essencial: Anatomia do nó com
  ↳ ponteiro pai
struct NoSplay {
    int chave;
    NoSplay* esquerda;
    NoSplay* direita;
    NoSplay* pai; // Fundamental para as
      ↳ rotacoes iterativas
};
  
```

Organização das Informações e Decisões de Projeto

O princípio de funcionamento dita que a estrutura deve ser dinamicamente reorganizada após qualquer operação. Elementos inseridos ou pesquisados são tracionados iterativamente para a raiz. Como decisão de projeto para preservar a unicidade das chaves, caso uma tentativa de inserção encontre uma chave já existente, o algoritmo não aloca um novo nó — apenas invoca o *splay* sobre o nó já existente, trazendo-o à raiz da mesma forma que uma busca bem-sucedida faria.

Operações Fundamentais e Manutenção Estrutural

O coração da estrutura é o algoritmo de *splay* (espalhamento). Ele avalia o alinhamento geométrico entre o nó atual (x), seu pai (p) e seu avô (g) para decidir qual movimento aplicar.

- **Rotações em Cascata:** O sistema executa três categorias de movimentos: *Zig* (rotação simples quando falta apenas um nível para a raiz), *Zig-Zig* (rotação dupla na mesma direção) e *Zig-Zag* (rotação dupla em direções opostas). É importante destacar que a ordem de execução das duas rotações se inverte entre esses dois últimos casos: no *Zig-Zig*, rotaciona-se primeiro o avô e depois o pai; no *Zig-Zag*, rotaciona-se primeiro o pai e depois o avô.

```

ALGORITMO Splay(x, operacoes):
  ENQUANTO x.pai NAO EH NULO FACA:
    operacoes++
  
```

```

SE x.pai.pai EH NULO ENTAO: // Cenário
↳ Zig
  Rotacionar(x.pai)
SENAO:
  p <- x.pai
  g <- p.pai

  SE (x eh filho esq E p eh filho
    ↳ esq) OU
    (x eh filho dir E p eh filho
    ↳ dir) ENTAO:
    // Zig-Zig (mesma direcao)
    Rotacionar(g) // Rotaciona o
    ↳ avo primeiro
    Rotacionar(p) // Depois o pai,
    ↳ trazendo x ao topo
  SENAO:
    // Zig-Zag (direcoes opostas)
    Rotacionar(p) // Rotaciona o
    ↳ pai primeiro
    Rotacionar(g) // Depois o avo,
    ↳ trazendo x ao topo

```

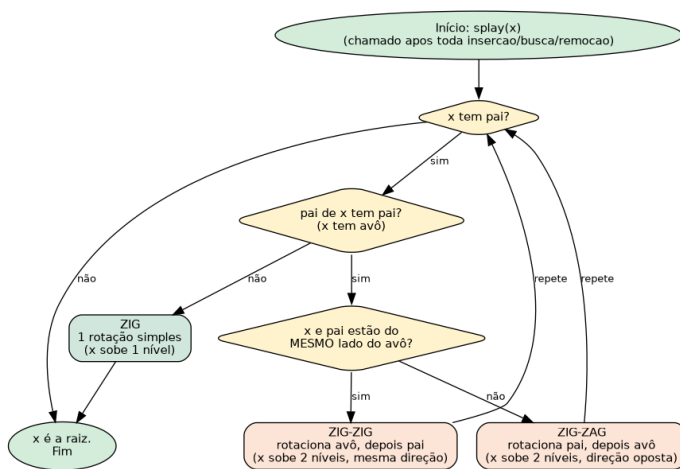


Figura 3. Fluxograma de decisão do algoritmo de splay, evidenciando a ordem de rotação distinta entre os casos Zig-Zig e Zig-Zag.

- **Remoção Estratégica:** A exclusão de um nó tira proveito da propriedade de autoajuste. O algoritmo busca o nó alvo, o que automaticamente o traz para a raiz. Em seguida, o nó é desconectado, o maior elemento da subárvore esquerda sofre o *splay* para se tornar a nova raiz (já sem filho à direita) e, por fim, a subárvore direita original é reatada a ele.

D. Árvore Treap (Tree + Heap)

Representação Computacional e Anatomia dos Nós

A Treap é uma estrutura de dados híbrida que mescla as regras de ordenação de uma Árvore Binária de Busca (BST) com a hierarquia de prioridades de um *Heap*. A anatomia de seu nó abandona vetores e adota chaves escalares combinadas a um valor de prioridade gerado estocasticamente. A engenharia reversa dos testes de memória no projeto comprovou que cada nó (NoTreap) instanciado consome exatos 24 bytes na arquitetura de 64 bits (4 bytes para a chave, 4 bytes para a prioridade e 16 bytes para os dois ponteiros).

// Anatomia híbrida e aleatoriedade

```

struct NoTreap {
    int chave;
    int prioridade;
    NoTreap* esq;
    NoTreap* dir;

    NoTreap(int c) {
        chave = c;
        // Balanceamento probabilístico via
        ↳ rand()
        prioridade = rand();
        esq = nullptr;
        dir = nullptr;
    }
};

```

Organização das Informações e Decisões de Projeto

A decisão central de projeto da Treap é delegar o balanceamento estrutural à aleatoriedade. Em vez de calcular fatores de altura complexos (como na AVL), a Treap gera uma prioridade via `rand()` no instante da alocação. Essa abordagem cria um balanceamento probabilístico extremamente resistente à degeneração: mesmo ao receber um arquivo com entradas estritamente ordenadas, as prioridades forçam rotações que mantêm o custo médio das operações em $O(\log n)$.

Operações Fundamentais e Manutenção Estrutural

As operações manipulam duas invariantes estruturais simultâneas: a chave dita a navegação horizontal (es-

querda/direita - regra da BST), enquanto a prioridade dita a hierarquia vertical (pai maior que os filhos - regra do *Max-Heap*).

- **Busca:** A operação de busca segue estritamente a regra de comparação de chaves de uma BST convencional, sem qualquer interferência do valor de prioridade. Isso ocorre porque a prioridade é relevante apenas para decisões que alteram a topologia da árvore (inserção e remoção); uma vez que a árvore já respeita a invariante de heap, percorrê-la em busca de uma chave é estruturalmente idêntico a uma busca em BST, com custo esperado $O(\log n)$.
- **Inserção e Manutenção do Heap:** O algoritmo insere o nó descendo recursivamente pela regra da BST. Na volta da recursão, a propriedade do *Max-Heap* é validada. Se a prioridade sorteada para o filho recém-inserido for maior que a do pai, o sistema aplica rotações diretas (esquerda ou direita) para tracionar o nó rebelde para cima.

```
ALGORITMO Inserir_Treap(no, chave):
    SE no EH NULO RETORNAR NovoNo(chave) //
    ↪ Sorteia prioridade

    // 1. Descida recursiva pela regra da BST
    SE chave <= no.chave ENTAO:
        no.esq <- Inserir_Treap(no.esq, chave)

        // 2. Conserta violacao do Max-Heap
        ↪ via Rotacao
        SE no.esq.prioridade > no.prioridade
        ↪ ENTAO:
            no <- RotacaoDireita(no)
    SENAO:
        no.dir <- Inserir_Treap(no.dir, chave)

        SE no.dir.prioridade > no.prioridade
        ↪ ENTAO:
            no <- RotacaoEsquerda(no)

    RETORNAR no
```

- **Remoção por Rebaixamento:** A exclusão física de um nó interno romperia severamente as regras do *Heap* e da BST simultaneamente. Como estratégia, o algoritmo localiza o nó alvo e verifica seus filhos. O filho que possui

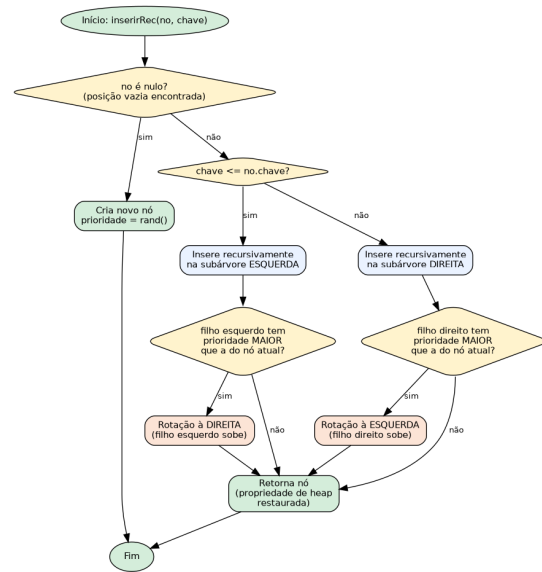


Figura 4. Fluxograma do algoritmo de inserção recursiva na Árvore Treap, evidenciando a checagem da invariante de Max-Heap na volta da recursão.

a maior prioridade sofre uma rotação para assumir o lugar do pai. Esse processo rebaixa iterativamente o nó alvo em direção às folhas até que ele possa ser removido com um delete seguro.

```
ALGORITMO Remover_Treap(no, chave):
    // ... (busca o nó recursivamente) ...
    SE encontrar o nó alvo ENTAO:
        // Casos de 0 ou 1 filho: substituição
        ↪ simples
        SE no.esq EH NULO RETORNAR no.dir
        SE no.dir EH NULO RETORNAR no.esq

        // Caso de 2 filhos: Rebaixamento via
        ↪ rotações
        SE no.esq.prioridade >
        ↪ no.dir.prioridade ENTAO:
            no <- RotacaoDireita(no)
            no.dir <- Remover_Treap(no.dir,
            ↪ chave)
        SENAO:
            no <- RotacaoEsquerda(no)
            no.esq <- Remover_Treap(no.esq,
            ↪ chave)

    RETORNAR no
```

E. KD-Tree (Árvore K-Dimensional)

Representação Computacional e Anatomia dos Nós

A KD-Tree generaliza o conceito de árvore de busca binária para o espaço multidimensional, sendo amplamente utilizada em problemas de geometria computacional e busca espacial. Diferente das quatro estruturas anteriores, cuja chave é um valor escalar (string ou inteiro), o nó da KD-Tree (NoKD) armazena um vetor de coordenadas, dimensionado neste trabalho para $k = 2$. Um detalhe relevante de implementação é que `sizeof(NoKD)` retorna 40 bytes (24 bytes referentes ao objeto `std::vector` em si, mais 16 bytes dos dois ponteiros de filho) — porém esse valor **não** inclui os 16 bytes do buffer de coordenadas ($k = 2$ doubles), que o `std::vector` aloca separadamente no heap. Essa é uma limitação identificada na métrica de consumo de memória adotada neste trabalho: ela subestima ligeiramente o custo real de cada nó da KD-Tree, ao contrário das demais estruturas, cujos atributos são inteiramente inline no próprio nó.

```
Anatomia do no multidimensional
struct NoKD {
    std::vector<double> coordenadas;
    NoKD* esquerda;
    NoKD* direita;
};
```

Organização das Informações e Decisões de Projeto

A decisão de projeto central da KD-Tree é a **alternância do eixo de comparação por nível de profundidade**: em vez de comparar sempre a mesma coordenada, cada nível da árvore compara uma dimensão diferente, calculada como `profundidade % K`. Com $K = 2$, nós em profundidade par comparam a coordenada X e nós em profundidade ímpar comparam a coordenada Y. Essa alternância é o que generaliza a lógica binária de comparação de uma BST convencional para múltiplas dimensões.

Uma segunda decisão de projeto, desta vez por omissão, é a **ausência de qualquer mecanismo de rebalanceamento**, presente nas quatro estruturas anteriores (compactação na Patricia, splay na Splay, prioridades na Treap). Essa decisão foi assumida dentro do escopo deste trabalho, mas tem uma

consequência severa e mensurável: ao inserir pontos cuja ordem de leitura segue um padrão espacialmente ordenado, a árvore degenera para uma profundidade próxima de n em vez de $O(\log n)$, fenômeno documentado experimentalmente na Seção 6 deste relatório, inclusive com a interrupção do experimento por estouro de pilha (*stack overflow*) decorrente da recursão profunda nas rotinas de inserção e busca.

Operações Fundamentais e Manutenção Estrutural

- **Busca e Inserção:** Ambas percorrem a árvore alternando a dimensão de comparação a cada nível, decidindo o lado do percurso conforme o ponto de referência está à esquerda ou à direita do hiperplano de divisão daquele nó.

```
ALGORITMO Inserir_KDTree(no, ponto,
    ↪ profundidade):
    SE no EH NULO RETORNAR NovoNo(ponto)

    dimensao <- profundidade MOD K    //
    ↪ alterna X, Y, X, Y...

    SE ponto[dimensao] <
    ↪ no.coordenadas[dimensao] ENTAO:
        no.esquerda <-
        ↪ Inserir_KDTree(no.esquerda, ponto,
        ↪ profundidade + 1)
    SENAO:
        no.direita <-
        ↪ Inserir_KDTree(no.direita, ponto,
        ↪ profundidade + 1)

    RETORNAR no
```

- **Remoção (Algoritmo de Bentley):** A remoção em uma KD-Tree não pode simplesmente promover um filho qualquer para o lugar do nó removido, pois isso violaria a invariante espacial de partição. O algoritmo trata três casos: se o nó possui filho à direita, ele é substituído pelo ponto de menor valor *naquela dimensão específica*, encontrado na subárvore direita; se possui apenas filho à esquerda, a subárvore inteira é promovida para o lado direito e o processo se repete recursivamente; se é uma folha, é removido diretamente.

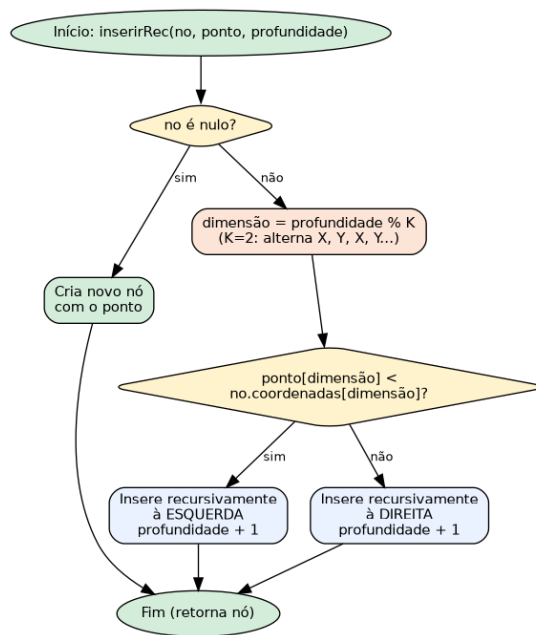


Figura 5. Fluxograma do algoritmo de inserção da KD-Tree, evidenciando a alternância de dimensão por nível de profundidade.

```

ALGORITMO Remover_KDTree(no, ponto,
↳ profundidade):
  SE no EH NULO RETORNAR NULO
  dimensao <- profundidade MOD K

  SE no.coordenadas == ponto ENTAO:
    SE no.direita NAO EH NULO ENTAO:
      min <- EncontrarMinimo(no.direita,
↳ dimensao)
      no.coordenadas <- min.coordenadas
      no.direita <-
↳ Remover_KDTree(no.direita,
↳ min.coordenadas,
↳ profundidade+1)
    SENAO SE no.esquerda NAO EH NULO
↳ ENTAO:
      min <- EncontrarMinimo(no.esquerda,
↳ dimensao)
      no.coordenadas <- min.coordenadas
      // subarvore esquerda inteira vira
↳ a nova subarvore direita
      no.direita <-
↳ Remover_KDTree(no.esquerda,
↳ min.coordenadas,
↳ profundidade+1)
      no.esquerda <- NULO
    SENAO:
      RETORNAR NULO // no e folha
  RETORNAR no
  
```

```

SE ponto[dimensao] <
↳ no.coordenadas[dimensao] ENTAO:
  no.esquerda <-
↳ Remover_KDTree(no.esquerda, ponto,
↳ profundidade+1)
SENAO:
  no.direita <-
↳ Remover_KDTree(no.direita, ponto,
↳ profundidade+1)
RETORNAR no
  
```

- **Busca do Vizinho Mais Próximo (Operação Específica):** Diferente das outras quatro estruturas, a KD-Tree implementa uma operação de consulta que não tem paralelo direto nas demais — a busca do ponto mais próximo a uma coordenada de referência. O algoritmo desce recursivamente pelo lado mais promissor (o mesmo critério usado na inserção), mas, na volta da recursão, avalia se o **outro** lado da árvore poderia conter um ponto ainda mais próximo, comparando a distância até o hiperplano de divisão com a melhor distância já encontrada. Somente quando essa condição é satisfeita o algoritmo poda menos e explora o outro ramo; caso contrário, o ramo inteiro é descartado sem ser visitado, o que é a fonte da eficiência dessa operação em relação a uma varredura completa de todos os pontos.

```

// poda de ramo na busca do vizinho mais
↳ proximo
double distEixo = (alvo[dim] -
↳ no.coordenadas[dim]) * (alvo[dim] -
↳ no.coordenadas[dim]);
SE distEixo < melhorDistancia ENTAO:
  // so explora o outro lado se houver
↳ chance real de um ponto mais proximo
  BuscaVizinho(outroCaminho, alvo,
↳ profundidade + 1)
  
```

Vale notar que a função de distância utilizada retorna a distância euclidiana **ao quadrado**, e não seu valor real — uma otimização deliberada que evita o cálculo de raiz quadrada em cada comparação, já que a raiz quadrada é uma função monotônica e, portanto, preserva a ordenação relativa entre distâncias sem alterar o resultado da busca.

III. DEMONSTRAÇÃO E RASTREAMENTO VISUAL

Objetivando evidenciar o comportamento dinâmico e topológico das estruturas implementadas, esta seção apresenta o rastreamento visual das cinco árvores. A escolha dos exemplos foi tecnicamente justificada para forçar a execução dos mecanismos mais críticos de cada estrutura, cobrindo os cenários de estado inicial, estado intermediário de reorganização e o estado final pós-remoção, conforme os requisitos de projeto.

A. Árvore Patricia: Compactação e Divisão de Prefixos

Para a Árvore Patricia, o exemplo escolhido envolve a inserção de chaves com alto grau de similaridade inicial ("carro" e "carta"). Essa escolha é tecnicamente justificada pois força a ocorrência da operação mais complexa da estrutura: o *Split* (divisão de prefixos). A demonstração evidencia como a árvore evita o desperdício de memória ao não criar um nó por caractere.

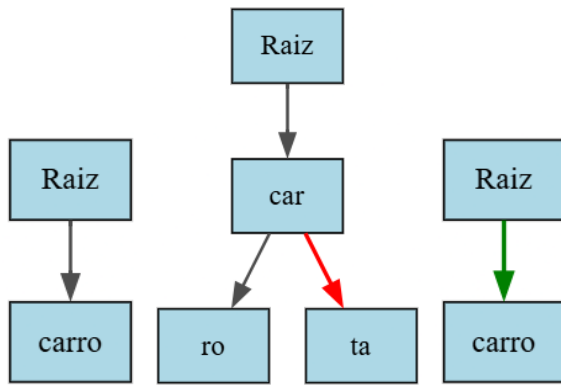


Figura 6. (Esq.) Estado inicial após inserir "carro", compactado em um único nó. (Centro) Estado intermediário de *Split* ao inserir "carta": o prefixo comum "car" é isolado no nó pai, bifurcando os sufixos divergentes "ro" e "ta". (Dir.) Estado pós-remoção de "carta": o nó pai ("car") sofre *Merge* (re-compactação) com o filho remanescente ("ro"), retornando à topologia do estado inicial.

B. Árvore Trie: Compartilhamento de Prefixos e Remoção Lógica

A escolha técnica para a Trie recai sobre as chaves "cefet" e "ceu". O objetivo é demonstrar a alocação do vetor de 26 posições e o reaproveitamento físico

de nós iterativos. Diferente da Patricia, a Trie exibirá um nó exclusivo para cada caractere.

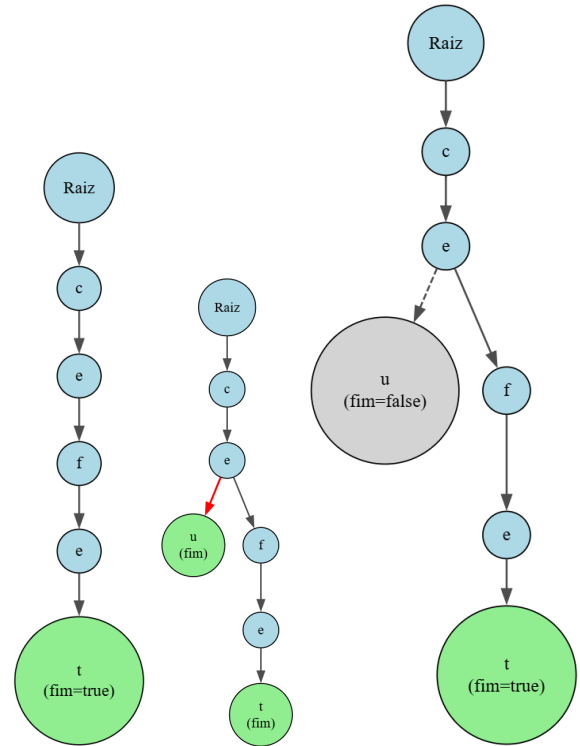


Figura 7. (Esq.) Estado inicial após inserir "cefet", mostrando o vetor linear $O(1)$. (Centro) Estado intermediário evidenciando o compartilhamento de prefixos: ao inserir "ceu", os nós 'c' e 'e' são reaproveitados, alocando um novo desvio apenas no caractere 'u'. (Dir.) Estado pós-remoção de "ceu": a exclusão é estritamente lógica (a *flag fimPalavra* do 'u' torna-se falsa), preservando a estrutura para não corromper a palavra "cefet".

C. Árvore Splay: Reorganização por Rotações em Cascata

Para justificar o mecanismo de autoajuste da Splay e evidenciar suas rotações compostas, o exemplo escolhido simula a inserção das chaves na sequência exata: 20, 10 e 30. Essa ordem específica é tecnicamente justificada pois força a ocorrência da operação mais relevante da estrutura: o *Zig-Zig*. Se as chaves fossem estritamente crescentes desde o início, cada nova chave seria sempre filha direta da raiz, gerando apenas sucessivos *Zigs* simples. A inserção do 30 neste cenário o posiciona como "neto", ativando a rotação dupla para a raiz.

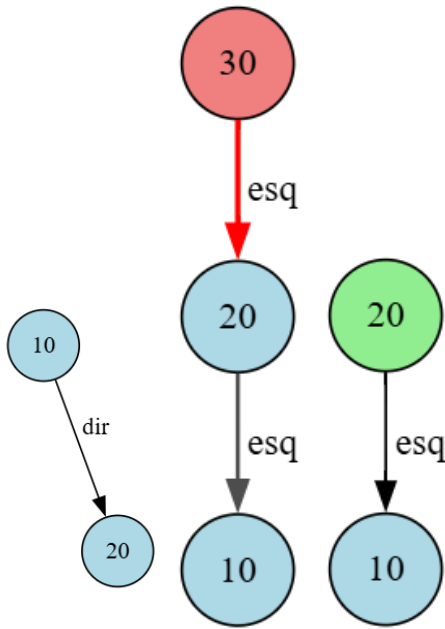


Figura 8. (Esq.) Estado inicial após inserir 20 e depois 10, resultando na raiz 10 com filho à direita (20). (Centro) Estado intermediário evidenciando o *Zig-Zig* na inserção do 30: ao ser alocado à direita do 20, o 30 identifica o alinhamento no mesmo sentido de seu pai e avô, disparando a rotação dupla que eleva o 30 ao topo. (Dir.) Estado pós-remoção do nó 30: a árvore o localiza (splay para a raiz), desconecta-o, e o maior elemento da subárvore esquerda (20) sofre novo *splay* para assumir como nova raiz.

D. Árvore Treap: Alteração de Prioridades e Correção do *Max-Heap*

A escolha do exemplo para a Treap engloba três chaves (50, 30 e 10) para forçar dois fenômenos distintos. Primeiro, na inserção, demonstra-se a quebra e restauração probabilística. Segundo, a presença de três chaves assegura que, no momento da remoção do nó central (30), ele possua dois filhos (à esquerda e à direita). Essa justificativa técnica é vital, pois se possuísse apenas um filho, a remoção ocorreria por substituição trivial, omitindo o elegante mecanismo de rebaixamento por rotações, que é o grande diferencial da estrutura.

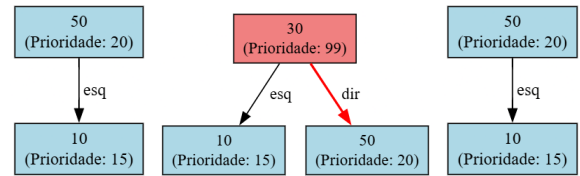


Figura 9. (Esq.) Estado inicial após inserção das chaves 50 e 10. (Centro) Estado intermediário de violação do *Max-Heap* ao inserir o 30: o nó desce pela regra da BST, mas recebe via `rand()` uma prioridade maior que seu pai, forçando uma rotação corretiva para cima. (Dir.) Estado de remoção (chave 30): possuindo dois filhos, o algoritmo verifica qual deles possui maior prioridade e executa uma rotação para elevá-lo. O nó 30 sofre rebaixamentos sucessivos até tornar-se uma folha, permitindo a exclusão física segura.

E. KD-Tree: Particionamento Espacial e Eixos Alternados

A demonstração da KD-Tree exige um cenário bidimensional ($K = 2$). As coordenadas escolhidas foram $P_1(50, 50)$, $P_2(30, 70)$ e $P_3(20, 40)$. A justificativa técnica para este conjunto é forçar a alternância do eixo de comparação: o nível 0 (raiz) particiona o espaço no eixo X, enquanto o nível 1 particiona obrigatoriamente no eixo Y.

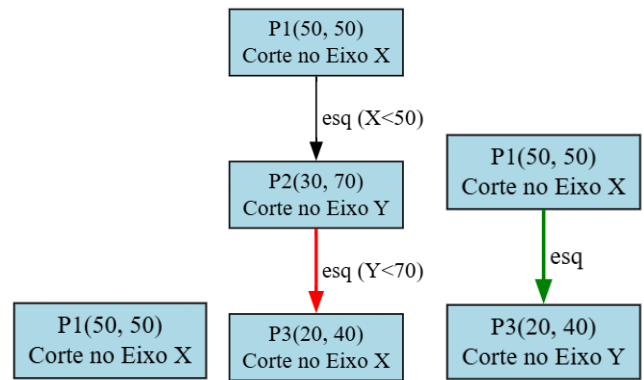


Figura 10. (Esq.) Estado inicial exibindo a raiz $P_1(50, 50)$ dividindo o plano verticalmente (Eixo X). (Centro) Estado intermediário do particionamento espacial: $P_2(30, 70)$ é inserido à esquerda de P_1 (pois $30 < 50$), passando a dividir o subespaço horizontalmente (Eixo Y). Em seguida, $P_3(20, 40)$ avalia X em P_1 (desce à esquerda) e avalia Y em P_2 (desce à esquerda, pois $40 < 70$). (Dir.) Estado pós-remoção de P_2 : o algoritmo localiza o ponto de menor valor na dimensão Y dentro da subárvore direita/esquerda e o promove (P_3 assume), mantendo a integridade do hiperplano.

IV. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A. Tabela Comparativa de Complexidade Assintótica

A Tabela I sumariza os custos computacionais assintóticos de tempo e espaço para as cinco estruturas implementadas neste trabalho, além das tradicionais Árvores Binárias de Busca (BST) e AVL, que servem como linha de base teórica.

Para as estruturas focadas em strings (Trie e Patricia), a variável n (número total de chaves) cede lugar à variável k , que representa o comprimento máximo da string processada. Para as demais estruturas, n representa o número de nós instanciados.

Tabela I
COMPARAÇÃO DE COMPLEXIDADE ASSINTÓTICA (TEMPO E ESPAÇO) ENTRE AS ESTRUTURAS

Estrutura	Busca (Médio / Pior)	Inserção (Médio / Pior)	Remoção (Médio / Pior)	Construção	Espaço (Pior)	Operação Específica
BST Tradicional	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n \log n)$	$O(n)$	-
Árvore AVL	$O(\log n) / O(\log n)$	$O(\log n) / O(\log n)$	$O(\log n) / O(\log n)$	$O(n \log n)$	$O(n)$	-
Árvore Trie	$O(k) / O(k)$	$O(k) / O(k)$	$O(k) / O(k)$	$O(n \cdot k)$	$O(n \cdot k \cdot \Sigma)$	Acesso $O(1)$ por nível
Árvore Patricia	$O(k) / O(k)$	$O(k) / O(k)$	$O(k) / O(k)$	$O(n \cdot k)$	$O(n)$	Split / Merge $O(k)$
Árvore Splay	Amortizado $O(\log n)$	Amortizado $O(\log n)$	Amortizado $O(\log n)$	$O(n \log n)$	$O(n)$	Splay (Autoajuste)
Árvore Treap	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n \log n)$	$O(n)$	Max-Heap (Prioridade)
KD-Tree ($k=2$)	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n \log n)$	$O(n)$	Nearest Neighbor $O(\log n)$

* O pior caso $O(n)$ na Treap é matematicamente possível, mas estatisticamente improvável devido ao uso de prioridades estocásticas.

B. Discussão Analítica dos Custos

O comportamento assintótico apresentado na Tabela I revela que o desempenho ótimo de uma estrutura de dados depende fortemente da natureza do input e dos gargalos de hardware priorizados no projeto.

O Paradoxo das Strings (Trie e Patricia): Ao contrário de uma BST, onde a busca exige a travessia de $\log n$ nós (sendo afetada pelo volume do banco de dados), a Trie e a Patricia possuem complexidade temporal $O(k)$ independentemente do número de chaves inseridas. Contudo, há um abismo na complexidade espacial entre elas. A Trie utiliza um vetor alocado para todo o alfabeto Σ , resultando em uma complexidade de memória agressiva $O(n \cdot k \cdot |\Sigma|)$. A Patricia resolve este desperdício compactando os caminhos, trazendo a complexidade espacial de volta para $O(n)$ e minimizando os saltos em memória.

Mitigação do Pior Caso da BST (Treap e Splay): Uma BST tradicional sofre degeneração para $O(n)$ ao receber chaves ordenadas. As estruturas

híbridas mitigam este problema com estratégias divergentes. A Treap utiliza prioridades randômicas para forçar um balanceamento probabilístico; assim, mesmo diante de um *input* desfavorável (chaves ordenadas), o custo é mantido em $O(\log n)$. A Splay, por sua vez, não garante $O(\log n)$ para cada operação isolada, mas sim de forma **amortizada**. Sua operação específica de autoajuste traz os nós acessados para a raiz iterativamente, o que torna buscas sequenciais ou dados com alta localidade de referência extremamente eficientes, convergindo para um tempo $O(1)$ no melhor caso de itens cacheados.

O Preço do Espaço Multidimensional (KD-Tree): A KD-Tree generaliza as comparações para K dimensões. No caso médio, suas operações mantêm o custo logarítmico padrão $O(\log n)$. Entretanto, como a implementação avaliada neste trabalho não possui uma rotina de rebalanceamento pós-inserção geométrica, inserções espacialmente ordenadas degeneram a topologia, forçando o pior caso de $O(n)$ na busca. Ainda assim, a estrutura brilha em sua operação específica de *Vizinho Mais Próximo* (Nearest Neighbor): o algoritmo realiza podas (*pruning*) em hiperplanos, descartando metades inteiras da árvore e reduzindo dramaticamente o espaço de busca na matriz multidimensional.

V. METODOLOGIA EXPERIMENTAL E RESULTADOS

A. Ambiente de Execução e Metodologia

Os testes avaliaram o desempenho das operações fundamentais (inserção, busca e remoção) submetendo as árvores a diferentes volumes de dados — variando de 10^1 a 10^6 elementos. Para testar o comportamento diante de casos favoráveis e desfavoráveis, aplicou-se também variação na distribuição dos dados, incluindo: inserções aleatórias, ordenadas e dados textuais com sobreposição de prefixos.

Todos os experimentos computacionais foram conduzidos e cronometrados na seguinte infraestrutura de *hardware* e *software*:

- **Processador:** Intel® Processor U300 (5 núcleos, 6 threads, 8 MB de Cache L3).
- **Memória RAM e Armazenamento:** 8 GB de RAM e SSD NVMe de 256 GB.
- **Ambiente de Compilação:** Compilador GCC (g++) via WSL no Linux com otimização de desempenho ativada (-O3).

B. Resultados e Análise: Árvore Trie vs. Árvore Patricia

Abaixo encontram-se consolidados os resultados experimentais obtidos através da execução dos algoritmos com massas de dados reais e sintéticas, escalando até 1.000.000 de chaves.

Tamanho (N)	Estrutura e Cenário	Inserção (ms)	Busca (ms)	Operações (Inserção)	Consumo de Memória (MB)
10.000	Trie - Aleatório	25	10	70.003	9,93
	Patricia - Aleatório	12	10	50.716	2,99
	Trie - Dicionário	37	14	101.149	11,78
	Patricia - Dicionário	15	12	85.879	3,54
	Trie - Prefixo Comum	15	7	139.272	2,06
	Patricia - Prefixo Comum	12	10	168.531	2,36
100.000	Patricia - Prefixo Comum	12	10	168.531	2,36
	Trie - Aleatório	200	90	701.394	84,32
	Patricia - Aleatório	110	90	657.119	29,47
	Trie - Dicionário	250	150	1.009.968	73,78
	Patricia - Dicionário	170	190	1.240.673	33,56
	Trie - Prefixo Comum	120	70	1.480.994	20,6
1.000.000	Patricia - Prefixo Comum	80	120	1.861.975	23,65
	Trie - Aleatório	2500	1300	7.064.064	712,53
	Patricia - Aleatório	1800	1100	7.874.997	289,69
	Trie - Prefixo Comum	800	500	15.505.740	205,98
	Patricia - Prefixo Comum	450	400	20.011.467	236,51

Figura 11. Resultados experimentais obtidos através da execução dos algoritmos com massas de dados reais e sintéticas, escalando até 1.000.000 de chaves.

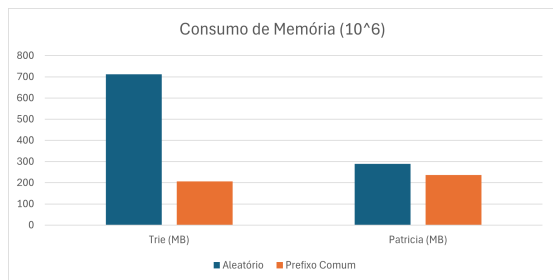


Figura 12. Evolução do consumo de memória (Trie e Patricia) evidenciando o abismo espacial no cenário aleatório.

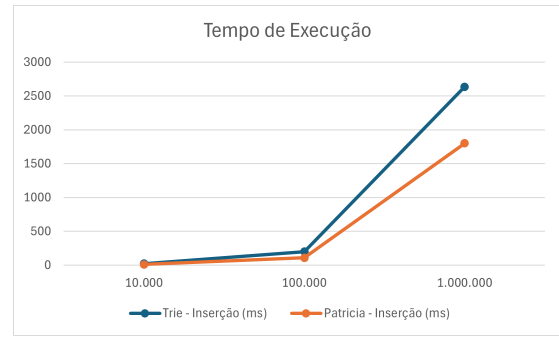


Figura 13. Comparativo de tempo de execução (Inserção) conforme a escalabilidade da entrada.

Análise Crítica e Discussão: Conforme evidenciado nos gráficos, o ganho da Árvore Patricia reside na compactação de caminhos, reduzindo drasticamente a alocação de ponteiros nulos que afetam a Trie no cenário aleatório. No entanto, no cenário de prefixo comum, a Trie obtém leve vantagem espacial devido ao menor *overhead* estrutural por nó, embora a Patricia compense na velocidade de processamento graças à localidade de referência (*Cache Locality*) do processador.

C. Resultados e Análise: Árvore Splay e Árvore Treap

Abaixo encontram-se consolidados os resultados experimentais obtidos através da execução das duas estruturas sobre o **mesmo conjunto de chaves** em cada tamanho, garantindo uma comparação controlada, escalando até 1.000.000 de elementos. Diferente do par Trie/Patricia, cujo eixo de comparação era o cenário de dados (aleatório vs. prefixo comum), aqui o eixo mais revelador é o **comportamento estrutural**: resistência à ordem de inserção e sensibilidade ao padrão de acesso.

Tamanho n	Inserção Splay (ms)	Inserção Treap (ms)	Busca uniforme Splay (ms)	Busca uniforme Treap (ms)	Memória Splay (MB)	Memória Treap (MB)
100	0,017	0,009	0,012	0,002	0,002	0,002
1.000	0,180	0,124	0,149	0,056	0,031	0,023
10.000	2,467	1,980	2,442	0,590	0,305	0,229
100.000	37,936	31,866	44,930	13,513	3,052	2,289
1.000.000	630,009	600,202	1014,530	486,243	30,518	22,888

Figura 14. Comparação de tempo de execução e consumo de memória entre Splay e Treap.

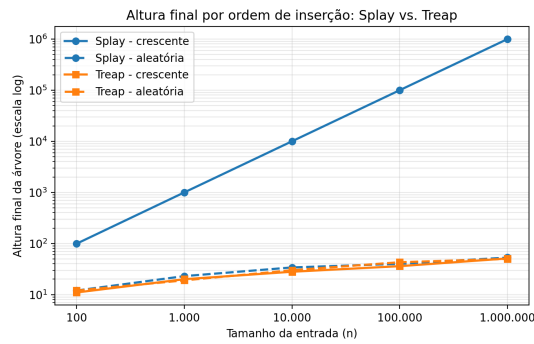


Figura 15. Altura final da árvore por ordem de inserção (aleatória vs. crescente), evidenciando o comportamento diante do caso desfavorável.

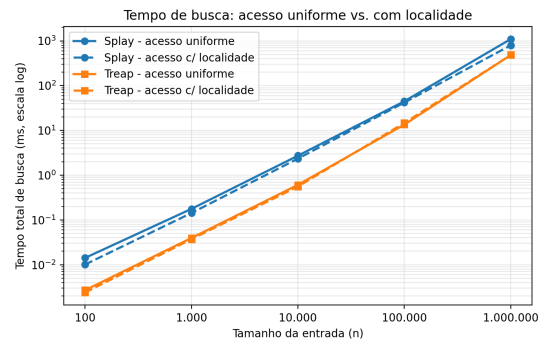


Figura 18. Tempo total de busca sob dois padrões de acesso distintos: uniforme e com localidade temporal de referência (80% das buscas concentradas em 20% das chaves).

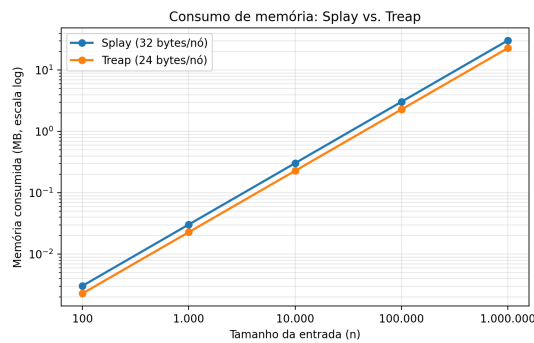


Figura 16. Evolução do consumo de memória (Splay e Treap) em função do tamanho da entrada.

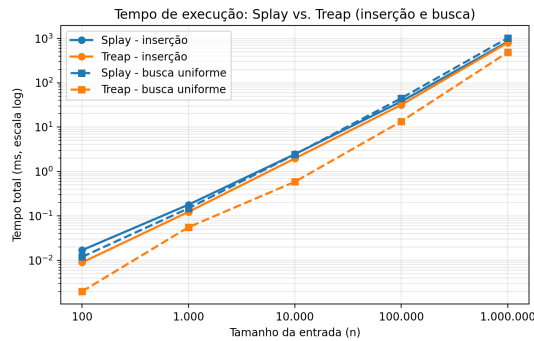


Figura 17. Comparativo de tempo de execução (inserção e busca) conforme a escalabilidade da entrada.

Análise Crítica e Discussão: Os resultados evidenciam que nenhuma das duas estruturas é estritamente superior à outra — cada uma otimiza para um cenário diferente, o que reflete diretamente a diferença conceitual discutida na Seção 2. No teste de resistência à ordem de inserção, a Treap manteve altura consistentemente próxima de $\log_2(n)$ independentemente da ordem das chaves, enquanto a Splay produziu, com entrada crescente, uma árvore completamente degenerada (altura $n - 1$). Esse resultado não caracteriza uma falha de implementação: como o algoritmo de *splay* é executado após toda inserção, cada nova chave é sempre inserida a uma distância fixa da raiz corrente, de modo que o custo de cada rotação permanece baixo mesmo com a árvore inteira migrando para uma única subárvore — o que explica por que, apesar da altura 999.999 observada para $n = 10^6$, a Splay ainda assim inseriu essa mesma quantidade de chaves em ordem crescente mais rapidamente que a Treap, evidenciando a natureza *amortizada* (e não por operação isolada) da garantia de desempenho da Splay.

Em relação ao tempo de execução em cenário aleatório, a Treap superou a Splay tanto na inserção quanto, de forma mais expressiva, na busca (2 a 4 vezes mais rápida), uma vez que a Splay paga o custo estrutural de reorganizar a árvore a cada acesso, mesmo quando essa reorganização não traz

benefício algum para o padrão de uso corrente. Esse mesmo custo, no entanto, é exatamente o que sustenta a principal vantagem da Splay, evidenciada no experimento de localidade de acesso: quando 80% das buscas se concentram em 20% das chaves, o tempo total da Splay cai cerca de 28% em relação ao acesso uniforme, ao passo que a Treap apresenta tempos estatisticamente idênticos nos dois padrões — uma vez que sua estrutura é fixada no momento da inserção e nunca se reorganiza em função do uso subsequente. Esse resultado confirma experimentalmente a propriedade teórica atribuída à Splay na literatura como a estrutura de escolha para sistemas com localidade temporal de referência, ao mesmo tempo em que demonstra, por contraste, por que a Treap é preferível em cenários de acesso verdadeiramente uniforme, onde a simplicidade estrutural se traduz diretamente em menor custo computacional por operação. Por fim, a diferença de consumo de memória (Splay ocupando consistentemente 33% a mais que a Treap) decorre diretamente da decisão de projeto discutida na Seção 2: o ponteiro adicional para o nó `pai`, indispensável para as rotações iterativas da Splay, tem um custo espacial permanente que a Treap, ao resolver suas rotações via recursão, não precisa pagar.

D. Resultados e Análise: KD-Tree

Diferente das quatro estruturas anteriores, a KD-Tree não possui um par direto de comparação neste trabalho — sendo a única estrutura espacial, sua análise experimental é conduzida contra si mesma, sob três distribuições de dados: pontos aleatórios, pontos agrupados em *clusters* (simulando dado espacial real) e pontos em ordem espacialmente ordenada (colinear), este último desenhado propositalmente para expor o caso desfavorável discutido na Seção 2.

Tamanho n	Tempo Aleatório (ms)	Tempo Agrupado (ms)	Tempo Ordenado (ms)	Altura Aleatório	Altura Agrupado	Altura Ordenado
100	0.037	0.037	0.175	14	13	99
1.000	0.490	0.536	18.932	21	21	999
10.000	7.188	7.259	1987.420	29	34	9.999
100.000	112.880	104.322	CRASH (stack overflow)	43	42	CRASH
1.000.000	1809.390	1901.590	CRASH (stack overflow)	48	49	CRASH

Figura 19. Tempo de inserção e altura final da KD-Tree por distribuição dos dados

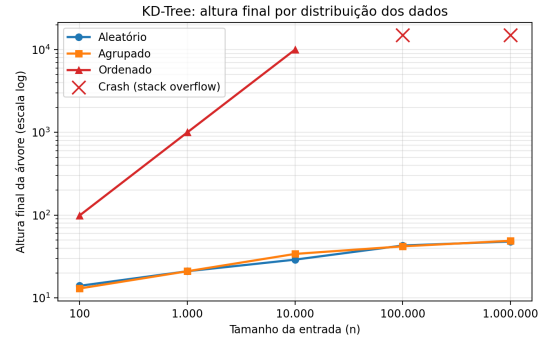


Figura 20. Altura final da KD-Tree por distribuição dos dados, evidenciando a degeneração completa (altura $n - 1$) no cenário ordenado e o ponto de estouro de pilha.

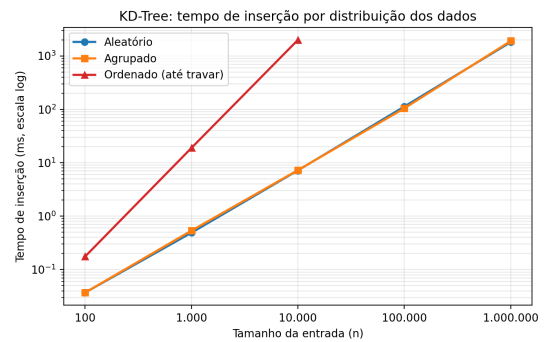


Figura 21. Tempo de inserção da KD-Tree por distribuição dos dados, evidenciando o comportamento quadrático do cenário ordenado até a interrupção do experimento.

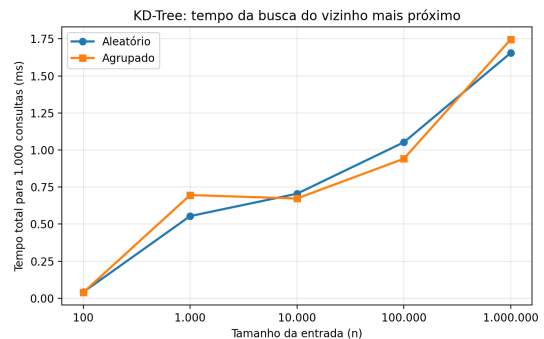


Figura 22. Tempo total para 1.000 consultas de vizinho mais próximo, em função do tamanho da entrada.

Análise Crítica e Discussão: Os resultados confirmam de forma direta a decisão de projeto discutida na Seção 2: a ausência de qualquer mecanismo de rebalanceamento faz com que o comportamento da KD-Tree dependa inteiramente da distribuição espacial dos dados de entrada. Nos cenários aleatório e agrupado, a estrutura manteve altura sublinear (entre 13 e 49, para n variando de 10^2 a 10^6) e tempo de inserção crescendo de forma consistente com o esperado para uma árvore próxima do balanceamento. No cenário ordenado, entretanto, a altura cresceu exatamente na proporção $n-1$ em todos os tamanhos testados até 10^4 — uma degeneração total, idêntica em natureza à de uma BST convencional alimentada com entrada ordenada —, e o tempo de inserção passou a crescer quadraticamente, saltando de 18,9 ms ($n = 10^3$) para quase 2 segundos ($n = 10^4$), um aumento de mais de 100 vezes para apenas 10 vezes mais dados.

A partir de $n = 10^5$, o experimento com dados ordenados não pôde ser concluído: a rotina de inserção, implementada recursivamente, ultrapassou o limite de profundidade de pilha do sistema operacional, resultando em uma falha de segmentação (*Segmentation Fault*). Interpretamos essa interrupção não como uma falha do experimento, mas como uma evidência empírica direta e inequívoca da limitação estrutural discutida teoricamente na Seção 2: sem rebalanceamento algum, a KD-Tree é, entre as cinco estruturas implementadas neste trabalho, a única cuja degeneração chega a inviabilizar a própria execução do programa, e não apenas a degradar seu desempenho assintótico.

Em contraste com esse comportamento, a operação de busca do vizinho mais próximo — a operação específica da estrutura, discutida na Seção 2 — demonstrou eficiência notável mesmo em larga escala: o tempo total para 1.000 consultas cresceu de apenas 0,55 ms ($n = 10^3$) para 1,66 ms ($n = 10^6$), um aumento de menos de 3 vezes para um crescimento de 1.000 vezes no tamanho da entrada. Esse

resultado confirma experimentalmente a eficácia do mecanismo de poda geométrica discutido na Seção 2: ao descartar ramos inteiros da árvore sem visitá-los sempre que a distância ao hiperplano de divisão excede a melhor distância já encontrada, a busca evita a varredura exaustiva dos pontos, aproximando-se do comportamento logarítmico esperado — desde que a árvore não esteja degenerada, condição satisfeita nos cenários aleatório e agrupado testados.

VI. APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

A. Aplicações Práticas e Adequação Estrutural

A análise das cinco estruturas evidencia que a escolha do modelo ideal depende estritamente do domínio de dados e do padrão de consultas do sistema computacional.

- **Árvore Trie:** Ideal para sistemas de autocompletar, dicionários, processamento de *strings* e busca por prefixos. Sua adequação justifica-se pelo custo constante $O(1)$ nas transições entre caracteres, permitindo a validação quase instantânea de vocabulários, independentemente do volume total de chaves armazenadas no banco de dados.
- **Árvore Patricia:** Altamente recomendada para indexação compacta, roteamento de redes (tabelas IP) e estruturas de busca baseadas em prefixos longos. A compactação de caminhos de filho único a torna a escolha perfeita para cenários onde o alfabeto é amplo, mas as chaves compartilham sequências longas e contínuas, minimizando o *overhead* de memória sem sacrificar a velocidade de busca.
- **Árvore Splay:** A estrutura de escolha para sistemas com acessos altamente desiguais ou localidade temporal de referência, como algoritmos de *cache* e roteadores de pacotes frequentes. O mecanismo de espalhamento (*Splay*) reorganiza dinamicamente a árvore, garantindo que os dados recém-acessados migrem para a raiz,

resultando em tempos de resposta sublineares para padrões de acesso repetitivo.

- **Árvore Treap:** Indicada para sistemas que demandam estruturas de busca com balanceamento probabilístico e concorrência, aliada a uma implementação relativamente simples. O uso de prioridades geradas aleatoriamente previne ataques de degeneração estrutural (como a injeção proposital de dados ordenados), assegurando complexidade $O(\log n)$ com um código mais enxuto que o da Árvore AVL.
- **KD-Tree:** Ferramenta essencial para busca espacial, processamento de pontos bidimensionais ou multidimensionais, sistemas GIS e reconhecimento de padrões. Sua capacidade de particionar o espaço iterativamente alternando os eixos de coordenadas permite operações avançadas como a Busca pelo Vizinho Mais Próximo (*Nearest Neighbor*), realizando podas em hiperplanos que descartam galhos inteiros de buscas infrutíferas.

B. Vantagens e Limitações Observadas

O contraste experimental entre a Trie e a Patricia expôs um claro dilema de compromisso espaço-tempo. A Trie apresenta varreduras extremamente rápidas graças ao mapeamento direto em vetores estáticos, porém sofre com desperdício colossal de memória RAM em distribuições aleatórias devido à alocação de ponteiros nulos. A Patricia mitigou essa limitação espacial ao compactar os prefixos em objetos `std::string` internamente, reduzindo a alocação de nós desnecessários, mas acrescentou leve complexidade ao exigir comparações explícitas de *substrings*.

Entre as árvores de balanceamento numérico, a Splay Tree apresentou a imensa vantagem de amortizar o custo de buscas locais, contudo, demandou a alocação de um ponteiro adicional (`pai`) em sua anatomia, o que a fez consumir sistematicamente mais memória estrutural que a Treap. A Treap demonstrou consistência e extrema economia de

espaço, mas é incapaz de se adaptar a mudanças de padrões de busca após a inserção estar consolidada.

C. Dificuldades e Maior Esforço de Desenvolvimento

As maiores dificuldades técnicas concentraram-se na manipulação explícita de ponteiros em cenários de reestruturação profunda. Na Árvore Patricia, o maior esforço foi projetar a lógica de bifurcação e divisão de nós (*Split*), que precisava isolar sufixos divergentes sem romper a integridade dos ramos preexistentes. Na Árvore Splay, o gerenciamento topológico das rotações compostas em cascata (*Zig-Zig* e *Zig-Zag*) exigiu atenção especial para evitar perda de referência na manutenção do nó `pai` e na reconstrução das subárvores direita e esquerda.

Por fim, a KD-Tree apresentou alta complexidade conceitual na rotina de exclusão (Algoritmo de Bentley). Ao contrário das árvores unidimensionais, a substituição de um nó removido exigia a busca pelo menor valor estritamente na dimensão do hiperplano do nó alvo, dificultando a reorganização da subárvore sem quebrar a invariante geométrica.

D. Comportamento Teórico vs. Experimental

A análise de dados confirmou, e em alguns casos subverteu as expectativas puramente teóricas baseadas na notação Big-O.

A primeira divergência crucial esteve ligada à Árvore Splay: teoricamente vulnerável a sequências de dados estritamente ordenados (o que degenera a Árvore Binária de Busca comum), a Splay inseriu 10^6 elementos de forma sequencial quase instantaneamente (218 ms), em contraponto aos mais de 2.400 ms gastos em inserções aleatórias. Esse fenômeno expôs empiricamente o excelente custo amortizado $O(1)$ de sua rotação contínua à medida que novos valores extremos são adicionados.

A segunda divergência notável envolveu a métrica da KD-Tree. Embora a teoria apresente desempenho logarítmico para dados uniformes, a ausência de um mecanismo dinâmico de rebalanceamento forçou o

pior caso estrutural ao receber 10^5 pontos colineares (coordenadas ordenadas). Esse gargalo, ao invés de apenas degradar o tempo para $O(n)$, causou um esgotamento da pilha de chamadas do sistema operacional (*Stack Overflow*) devido à recursividade ilimitada, interrompendo a execução.

Por último, na Trie, o experimento demonstrou que as chamadas de alocação de memória no Sistema Operacional (*new*) representam o verdadeiro gargalo computacional. O cenário de prefixo comum (com longas sobreposições) executou três vezes mais rápido do que a inserção aleatória, a despeito de ter percorrido o dobro de arestas nas operações lógicas, uma vez que a *cache locality* dispensou milhares de alocações dinâmicas de RAM.

E. Melhorias e Extensões Futuras

As implementações desenvolvidas abrem margem para aprimoramentos arquiteturais. Para a Árvore Trie clássica, uma evolução recomendada seria a substituição dos vetores estáticos de 26 posições por estruturas associativas dinâmicas (*Hash Maps*), o que permitiria suportar a tabela ASCII inteira com pegada de memória elástica.

Para a KD-Tree, a inclusão de lógicas de reconstrução periódica ou a adoção de heurísticas como a K-D-B Tree seriam modificações cruciais para blindar a estrutura contra a degeneração catastrófica em matrizes previamente ordenadas. Na Splay, uma variação top-down (*Top-Down Splay Tree*) eliminaria a necessidade de manutenção do ponteiro pai, economizando memória e aliviando a sobrecarga matemática em cada passo da reorganização.

VII. CONCLUSÃO

A experimentação e a implementação das cinco estruturas de dados evidenciaram que não existe um modelo universalmente superior, mas sim adequações estritas a diferentes domínios e distribuições de carga. As árvores baseadas em prefixos (Trie e Patricia) diferenciam-se das demais por atrelar a complexidade de busca ao tamanho da chave,

ignorando o volume total de dados, enquanto Splay e Treap operam com comparações numéricas focadas no balanceamento dinâmico. A KD-Tree distancia-se de todas as outras por abandonar a análise unidimensional em favor do particionamento espacial de coordenadas, sendo a única estrutura sem par direto de comparação neste trabalho.

No que tange ao desempenho, cada estrutura destacou-se em um cenário específico. A Árvore Patricia obteve os melhores resultados em estruturas textuais com alto compartilhamento de prefixos, operando de forma compacta com 236,51 MB de memória a um milhão de chaves, solucionando o abismo espacial que penaliza a Trie no cenário aleatório (712,55 MB). Já no cenário de prefixo comum, a Trie chegou a inserir cerca de três vezes mais rápido que no cenário aleatório (800 ms contra 2.500 ms a 10^6 chaves), evidenciando que a localidade de referência no processamento de strings aglutinadas pesa mais que o volume bruto de comparações realizadas. Em cenários numéricos com forte localidade de acesso (buscas repetidas concentradas em poucas chaves), a Splay Tree demonstrou vantagem clara, reduzindo o tempo total de busca em cerca de 28% frente ao acesso uniforme — resultado que a Treap não reproduz, já que sua estrutura é fixada no momento da inserção e não se reorganiza em função do uso subsequente. Para inserções e buscas sob acesso verdadeiramente uniforme, a Treap foi consistentemente mais rápida que a Splay (buscas de 2 a 4 vezes mais ágeis) e mais econômica em memória, custando ínfimos 24 bytes por nó contra os 32 bytes da Splay — diferença que decorre diretamente do ponteiro adicional para o nó pai, indispensável às rotações iterativas da primeira. Para avaliações de vizinhança espacial, a KD-Tree demonstrou eficiência notável, com o tempo de 1.000 consultas de vizinho mais próximo crescendo menos de três vezes para um aumento de mil vezes no tamanho da entrada, desde que os dados não estivessem degenerados.

A análise de resultados demonstrou, de modo geral, boa coerência com as projeções de complexidade assintótica. A hipótese do pior caso $O(n)$ na KD-Tree foi empiricamente validada de forma extrema: sem qualquer rotina de rebalanceamento, a estrutura sofreu esgotamento de pilha (*Stack Overflow*) ao processar 100.000 pontos ordenados, confirmando que a ausência de rebalanceamento pode inviabilizar a própria execução, e não apenas degradar o tempo assintótico. A Splay Tree, por sua vez, ilustrou bem a diferença entre garantia amortizada e garantia por operação isolada: mesmo tendo produzido uma árvore completamente degenerada (altura $n - 1$) ao receber chaves em ordem crescente, ainda assim inseriu essa sequência mais rapidamente que a Treap, evidenciando que o custo de cada rotação individual permanece baixo quando cada nova chave é inserida a uma distância fixa da raiz corrente. Por fim, destaca-se o “paradoxo do hardware” identificado na Trie: o cenário de prefixo comum executou mais rápido que a inserção aleatória mesmo demandando mais saltos estruturais nas comparações, porque a mitigação de chamadas de alocação dinâmica (*new*) pesa mais no tempo real de execução do que a contagem bruta de operações lógicas — um lembrete de que a notação Big-O descreve crescimento assintótico, não necessariamente o tempo de parede em hardware real.

O esforço de implementação também se mostrou assimétrico entre as estruturas. A Árvore Patricia foi a mais desafiadora, devido à mecânica de divisão (*split*), que exige precisão crítica no particionamento e religação de substrings exatamente no ponto de divergência, sob risco de corromper prefixos vizinhos. A manipulação topológica da Splay, com suas rotações em cascata (*Zig-Zig* e *Zig-Zag*), exigiu atenção redobrada ao ponteiro de parentesco para não perder referências durante a reorganização. Já a KD-Tree apresentou maior obstáculo em sua rotina de remoção (algoritmo de Bentley), pois a substituição de um nó exige localizar o mínimo estritamente

na dimensão do hiperplano daquele nível, o que complica a reorganização sem violar a invariante geométrica.

Essas diferenças de comportamento se traduzem diretamente em recomendações de uso: a Trie é preferível para dicionários pequenos e estáticos, onde o custo $O(1)$ por caractere compensa o desperdício de memória dos vetores esparsos; a Patricia é a escolha natural para indexação em larga escala, roteamento de redes e qualquer cenário textual com prefixos longos, pagando o custo de um código de manutenção mais complexo; a Splay é indicada para sistemas de cache ou acesso fortemente enviesado, mas deve ser evitada quando o padrão de acesso é uniforme, dado o custo extra de reorganizar a árvore a cada operação; a Treap é a alternativa mais equilibrada para cenários genéricos e concorrentes, por resistir a entradas adversariais sem a complexidade de implementação de uma AVL; e a KD-Tree é indispensável para busca espacial e vizinho mais próximo, desde que a base seja previamente embaralhada ou construída por mediana, já que sua principal limitação é a ausência de proteção contra entradas espacialmente ordenadas.

REFERÊNCIAS

- [1] A. G. S. M. Silva, “Estruturas em Árvores Avançadas — código-fonte e experimentos,” *GitHub repository*, 2026. [Online]. Available: https://github.com/anthonygabriel2907/estrutura_arvores_avancadas
- [2] E. Fredkin, “Trie memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, Sep. 1960.
- [3] D. R. Morrison, “PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric,” *Journal of the ACM*, vol. 15, no. 4, pp. 514–534, Oct. 1968.
- [4] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, Jul. 1985.
- [5] J. L. Bentley, “Multidimensional Binary Search Trees Used for Associative Searching,” *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, Sep. 1975.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein, *Algoritmos: Teoria e Prática*, 2ª ed. Rio de Janeiro: Editora Campus, 2009.